

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956) ENATHUR,
KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : P. Sri Manaswini
Reg No : 11249A356
Class : II B.E(CSE)
Course Code :
Course Name : Data Science Toolkit Lab.

**SRI CHANDRASEKHADRENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

**UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956) ENATHUR,
KANCHIPURAM - 631 561**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFAIDE CERTIFICATE

This is to certify that this is the Bonafide record of work done by

Ms. _____ P. Sri Manaswini _____

with Reg. No 11249A356 done by of II-B.E.(CSE) in DATA SCIENCE TOOLKIT for Lab
during the academic year 2025 -2026.

Station : Enathur

Date :

Staff-in-charge

Head of the department

Submitted for the Practical examination held on _____

Examine-1

Examine

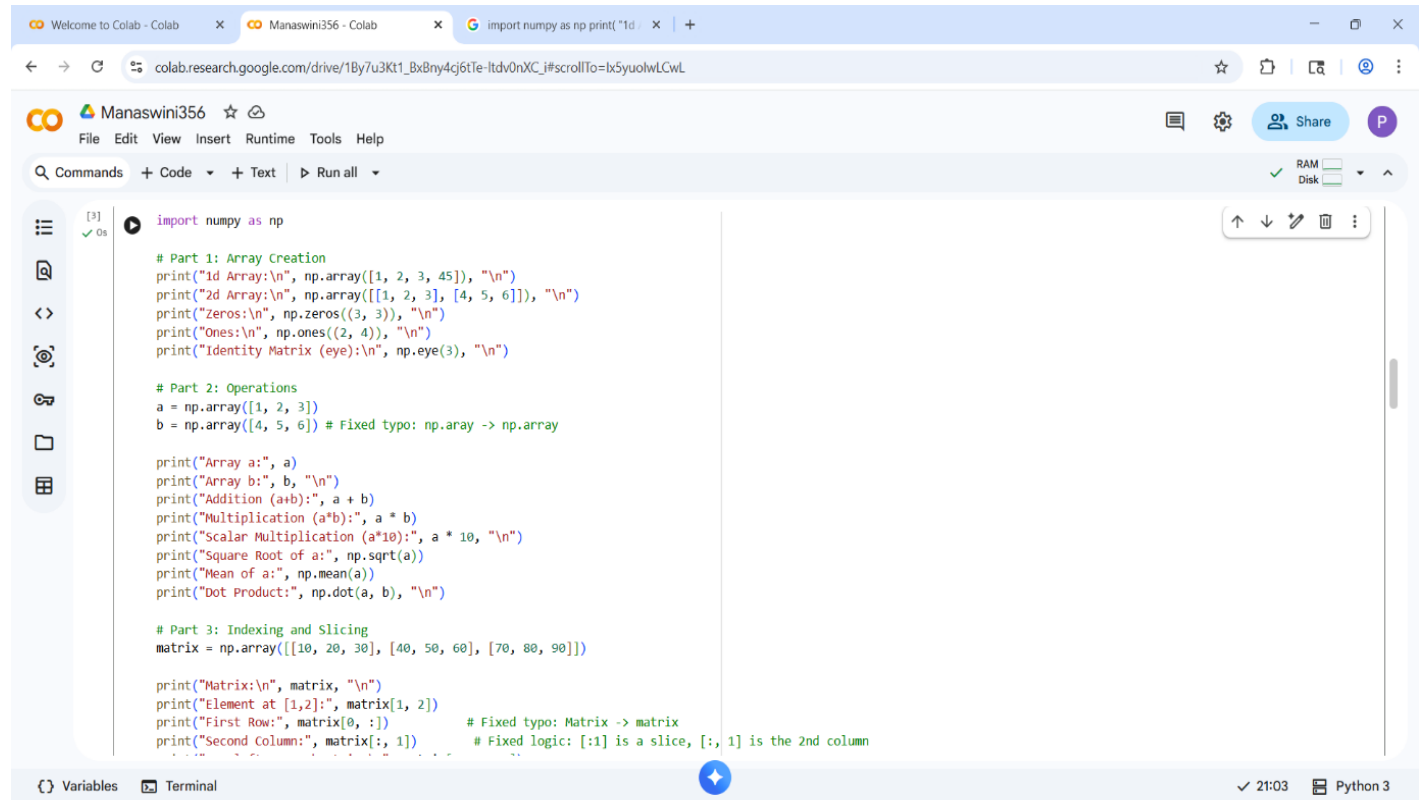
INDEX

SINO	NAME OF EXPERIMENT	DATE	PG NO
1	NumPy and Pandas	2-02-26	1
2	Matplotlib and Scikit Learn	9-02-26	5
2.1	Data Exploriation and Preprocess In Python	9-02-26	5
3	Implement Regularized Linear Regression	14-02-26	8
4	Implement Naïve Bayes Classifer	23-02-26	9
5	Implement Regularized Logistic Regression	2-03-26	10
6	Build Models uing different Ensembling Techniques	9-03-26	12
7	Build Models using Decision Tress	9-03-26	13
8	Kernel Functions	16-03-26	14
9	Implement K-Nearest Neighbors (KNN) Classification	23-03-26	15
10	K-Means Clustering with PCA and Elbow Method	23-03-26	16

EXP NO:1

AIM: Create Arrays ,Array Operations ,Indexing and Slicing

PROGRAM:



The screenshot shows a Google Colab notebook titled "Manaswini356". The code is organized into three parts: Part 1: Array Creation, Part 2: Operations, and Part 3: Indexing and Slicing. The code uses NumPy to create various arrays and perform operations like addition, multiplication, and slicing. The output of the code is visible in the terminal at the bottom.

```
import numpy as np

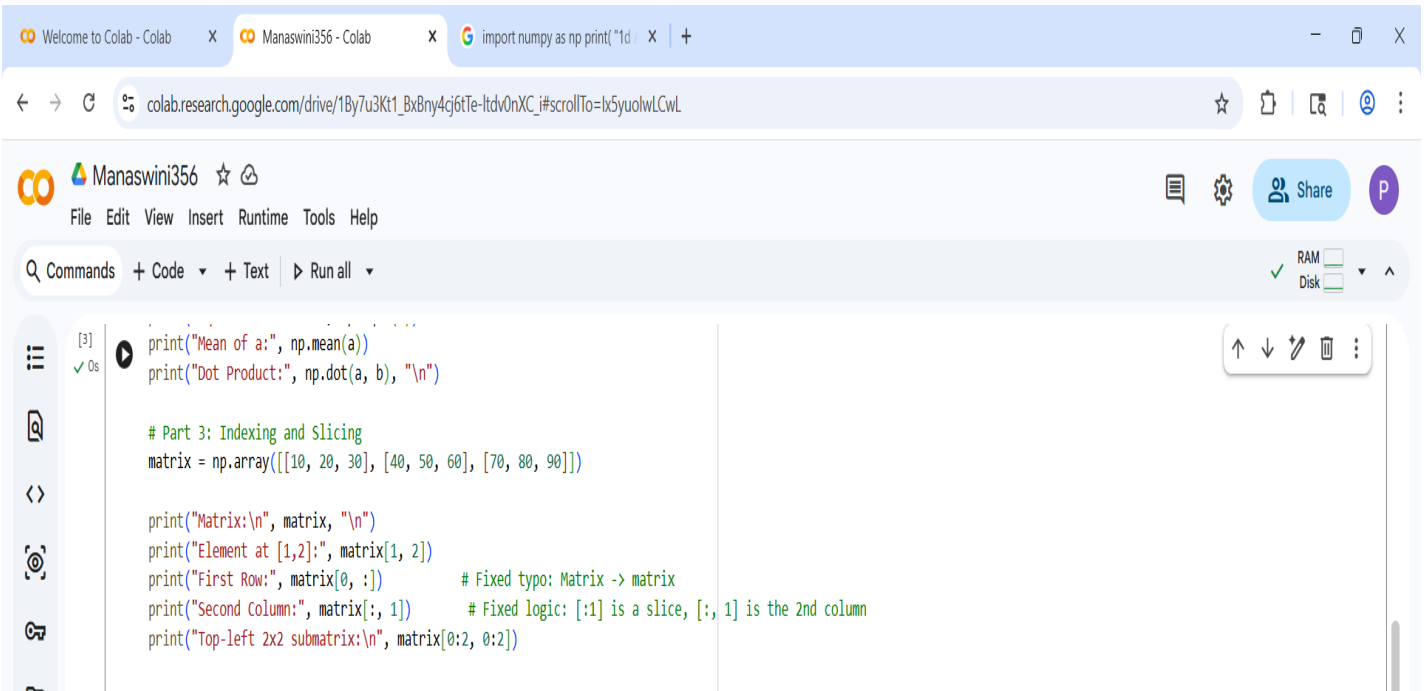
# Part 1: Array Creation
print("1d Array:\n", np.array([1, 2, 3, 45]), "\n")
print("2d Array:\n", np.array([[1, 2, 3], [4, 5, 6]]), "\n")
print("Zeros:\n", np.zeros((3, 3)), "\n")
print("Ones:\n", np.ones((2, 4)), "\n")
print("Identity Matrix (eye):\n", np.eye(3), "\n")

# Part 2: Operations
a = np.array([1, 2, 3])
b = np.array([4, 5, 6]) # Fixed typo: np.array -> np.array

print("Array a:", a)
print("Array b:", b, "\n")
print("Addition (a+b):", a + b)
print("Multiplication (a*b):", a * b)
print("Scalar Multiplication (a*10):", a * 10, "\n")
print("Square Root of a:", np.sqrt(a))
print("Mean of a:", np.mean(a))
print("Dot Product:", np.dot(a, b), "\n")

# Part 3: Indexing and Slicing
matrix = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

print("Matrix:\n", matrix, "\n")
print("Element at [1,2]:", matrix[1, 2])
print("First Row:", matrix[0, :]) # Fixed typo: Matrix -> matrix
print("Second Column:", matrix[:, 1]) # Fixed logic: [:1] is a slice,[:, 1] is the 2nd column
```



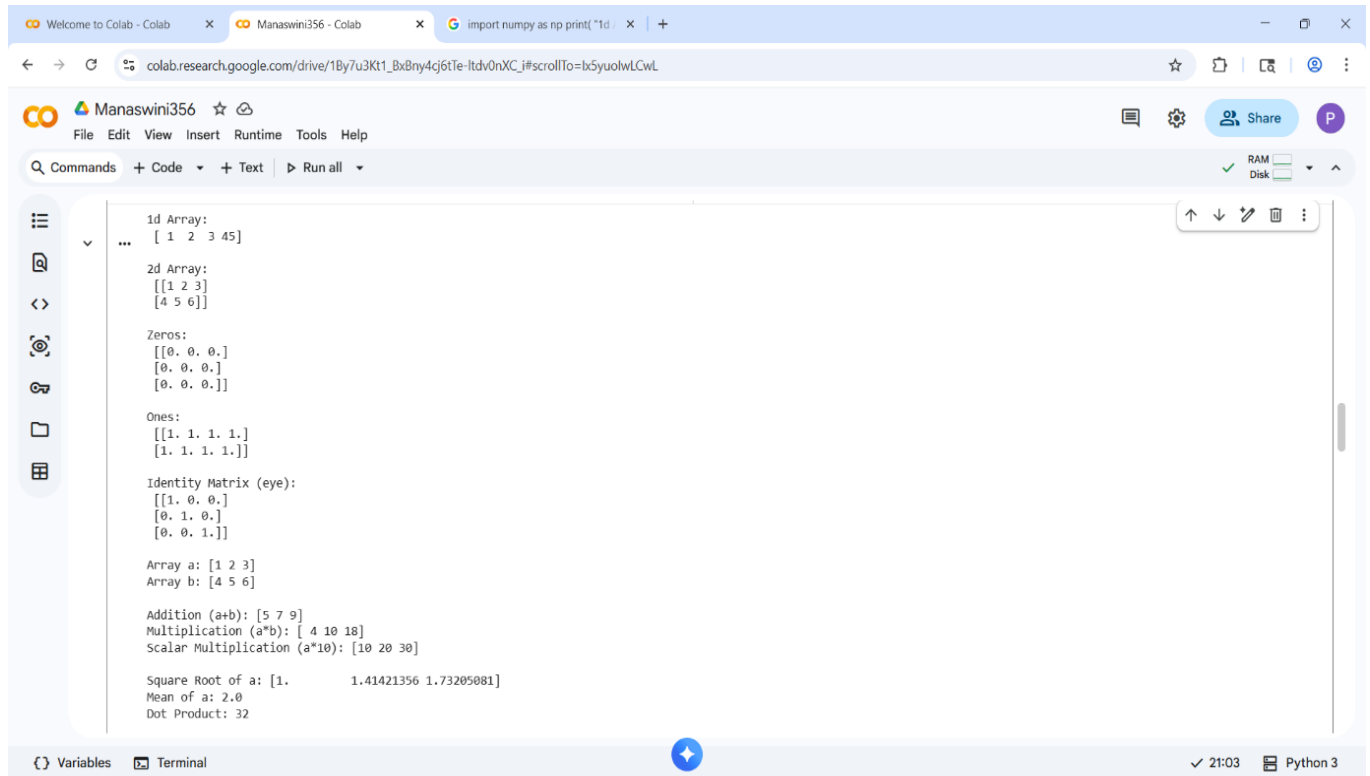
This screenshot shows the continuation of the Python program from the previous screenshot. It includes the final part of the code for indexing and slicing, and the output of the program is visible in the terminal.

```
print("Mean of a:", np.mean(a))
print("Dot Product:", np.dot(a, b), "\n")

# Part 3: Indexing and Slicing
matrix = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

print("Matrix:\n", matrix, "\n")
print("Element at [1,2]:", matrix[1, 2])
print("First Row:", matrix[0, :]) # Fixed typo: Matrix -> matrix
print("Second Column:", matrix[:, 1]) # Fixed logic: [:1] is a slice,[:, 1] is the 2nd column
print("Top-left 2x2 submatrix:\n", matrix[0:2, 0:2])
```

OUTPUT:



The image shows a Google Colab notebook interface. The top bar includes the Colab logo, the name 'Manaswini356', and a 'Share' button. Below the top bar is a menu with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar contains 'Commands', '+ Code', '+ Text', and 'Run all'. The main code cell contains the following output:

```
1d Array:  
...  
[ 1  2  3 45]  
  
2d Array:  
[[1 2 3]  
 [4 5 6]]  
  
Zeros:  
[[0. 0. 0.]  
 [0. 0. 0.]  
 [0. 0. 0.]]  
  
Ones:  
[[1. 1. 1. 1.]  
 [1. 1. 1. 1.]]  
  
Identity Matrix (eye):  
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]  
  
Array a: [1 2 3]  
Array b: [4 5 6]  
  
Addition (a+b): [5 7 9]  
Multiplication (a*b): [ 4 10 18]  
Scalar Multiplication (a*10): [10 20 30]  
  
Square Root of a: [1.          1.41421356 1.73205081]  
Mean of a: 2.0  
Dot Product: 32
```

The bottom status bar shows 'Variables', 'Terminal', a blue circular icon, '21:03', and 'Python 3'.



The image shows the same Google Colab notebook interface, but the code cell now displays the second part of the output:

```
Scalar Multiplication (a*10): [10 20 30]  
  
Square Root of a: [1.          1.41421356 1.73205081]  
Mean of a: 2.0  
Dot Product: 32  
  
Matrix:  
[[10 20 30]  
 [40 50 60]  
 [70 80 90]]  
  
Element at [1,2]: 60  
First Row: [10 20 30]  
Second Column: [20 50 80]  
Top-left 2x2 submatrix:  
[[10 20]  
 [40 50]]
```

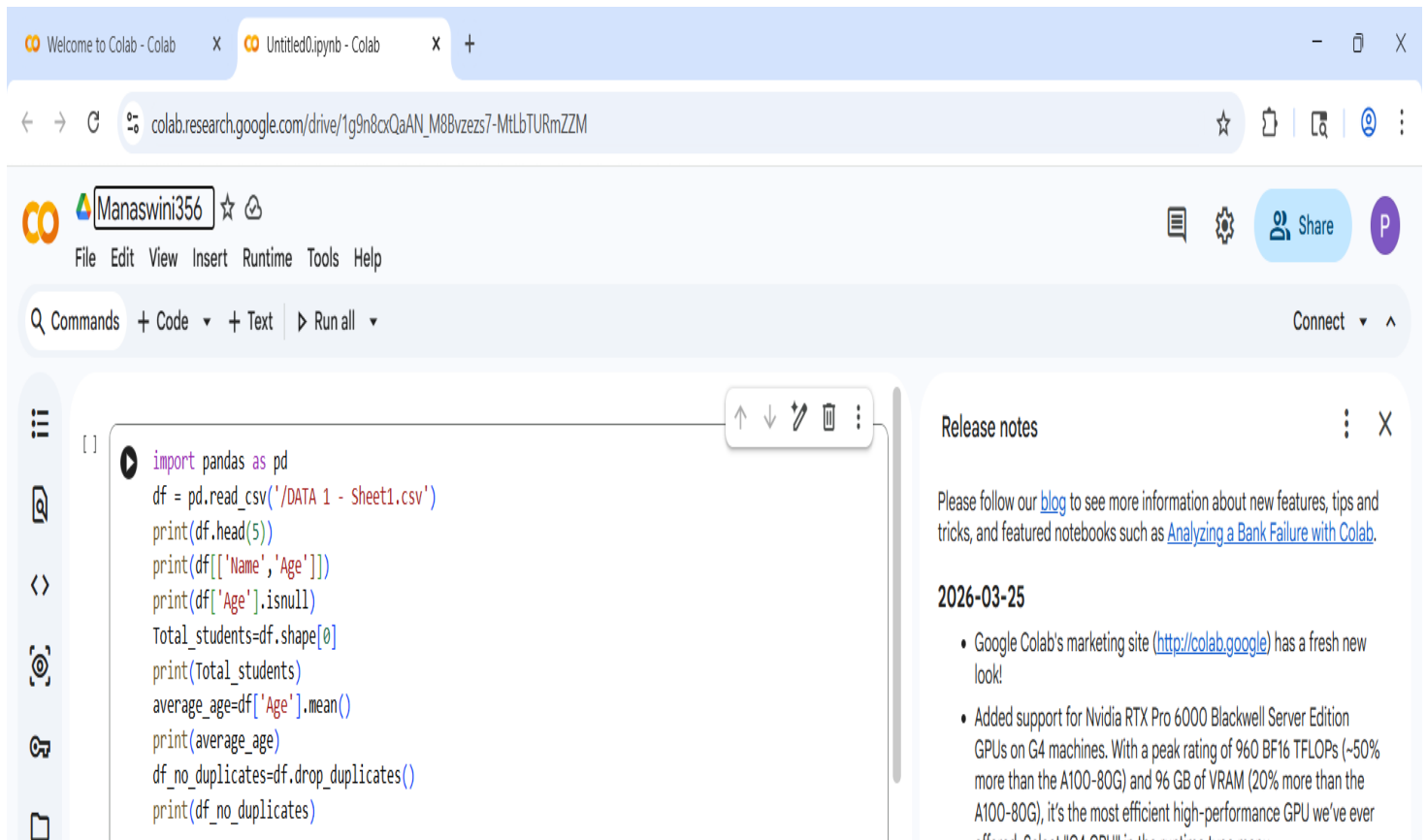
RESULT:

Basic NumPy operations were performed successfully

EXP: 1.1

AIM: Creating Pandas Operations

PROGRAM:



The screenshot displays a Google Colab notebook environment. The browser address bar shows the URL: `colab.research.google.com/drive/1g9n8cxQaAN_M8Bvzezs7-MtLbTURmZZM`. The notebook interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with icons for commands, code, text, and running all cells. The main area contains a code cell with the following Python code:

```
[ ] import pandas as pd
df = pd.read_csv('/DATA 1 - Sheet1.csv')
print(df.head(5))
print(df[['Name', 'Age']])
print(df['Age'].isnull)
Total_students=df.shape[0]
print(Total_students)
average_age=df['Age'].mean()
print(average_age)
df_no_duplicates=df.drop_duplicates()
print(df_no_duplicates)
```

On the right side of the notebook, there is a 'Release notes' panel. It contains the following text:

Please follow our [blog](#) to see more information about new features, tips and tricks, and featured notebooks such as [Analyzing a Bank Failure with Colab](#).

2026-03-25

- Google Colab's marketing site (<http://colab.google>) has a fresh new look!
- Added support for Nvidia RTX Pro 6000 Blackwell Server Edition GPUs on G4 machines. With a peak rating of 960 BF16 TFLOPs (~50% more than the A100-80G) and 96 GB of VRAM (20% more than the A100-80G), it's the most efficient high-performance GPU we've ever offered. Select "G4 CPU" in the runtime type menu.

OUTPUT:

The image displays two screenshots of Google Colab notebooks. The top screenshot shows a notebook titled 'Untitled0.ipynb' with the following code and output:

```
print(average_age)
df_no_duplicates=df.drop_duplicates()
print(df_no_duplicates)
```

The output shows a DataFrame with columns: Register No, Name, Age, Department, and Section. The data is as follows:

Register No	Name	Age	Department	Section
0	E1	20	CSE	A
1	Mike	21	CSE	B
2	Max	22	CSE	C
3	Eddie	23	CSE	D
4	Dustin	24	CSE	E

The bottom screenshot shows a notebook titled 'Manaswini35d' with the following code and output:

```
<bound method Series.isnull of 0 20
1 21
2 22
3 23
4 24
5 25
6 26
7 27
8 38
9 29
Name: Age, dtype: int64>
```

The output shows a DataFrame with columns: Register No, Name, Age, Department, and Section. The data is as follows:

Register No	Name	Age	Department	Section
0	E1	20	CSE	A
1	Mike	21	CSE	B
2	Max	22	CSE	C
3	Eddie	23	CSE	D
4	Dustin	24	CSE	E
5	Lucas	25	CSE	F
6	Holly	26	CSE	G
7	Will	27	CSE	H
8	Steve	38	CSE	I
9	Nancy	29	CSE	J

On the right side of the bottom screenshot, there is a 'Release notes' section with the following content:

Release notes

Please follow our [blog](#) to see more information about new features, tips and tricks, and featured notebooks such as [Analyzing a Bank Failure with Colab](#).

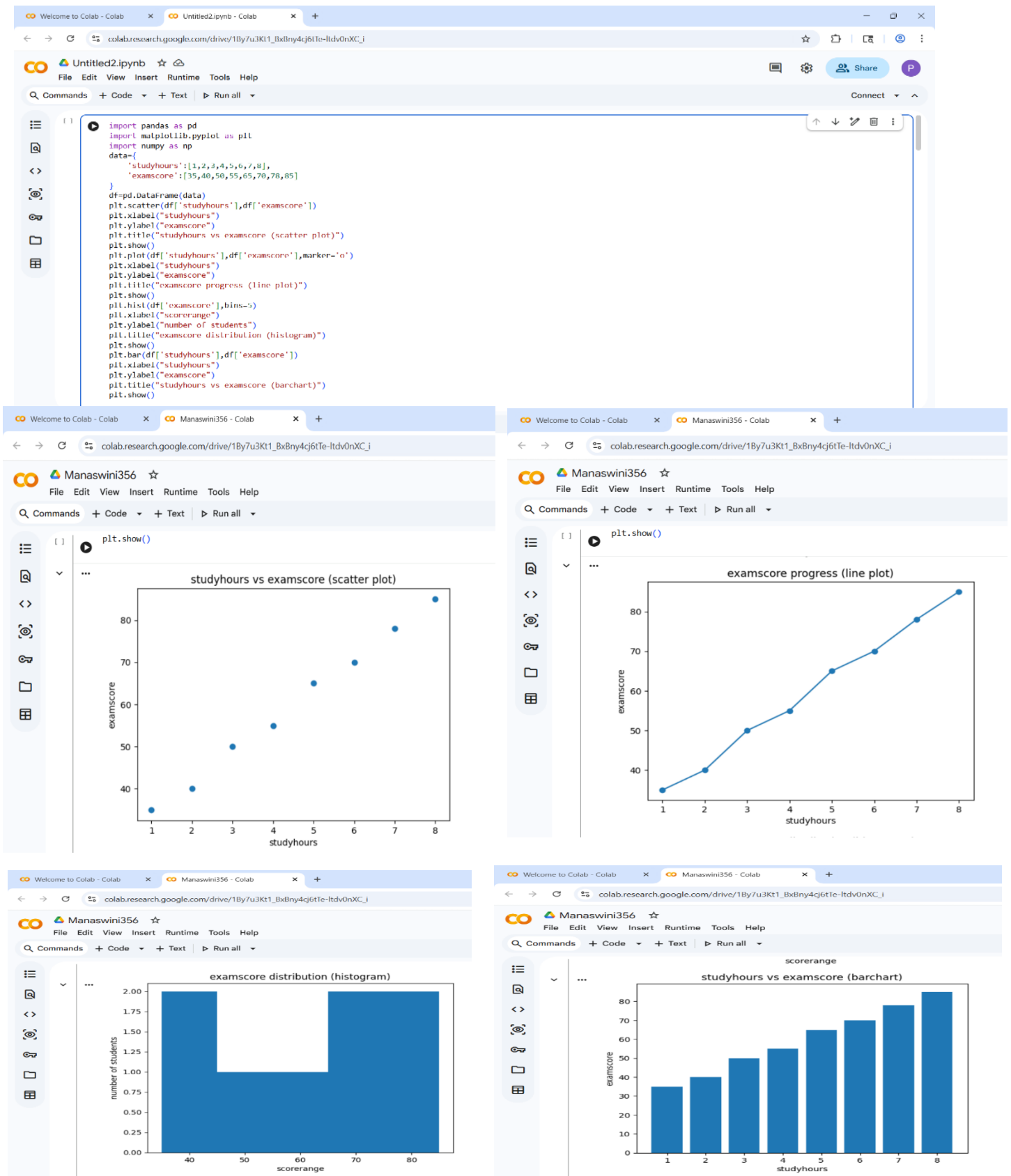
2026-03-25

- Google Colab's marketing site (<http://colab.google>) has a fresh new look!
- Added support for Nvidia RTX Pro 6000 Blackwell Server Edition GPUs on G4 machines. With a peak rating of 960 BF16 TFLOPs (~50% more than the A100-80G) and 96 GB of VRAM (20% more than the A100-80G), it's the most efficient high-performance GPU we've ever offered. Select "G4 GPU" in the runtime type menu.
- Colab's new Open Source MCP Server gives your local agents programmatic access to control the entire notebook development lifecycle. Use your local agent to build, run, and visualize in the browser. ([GitHub Repo](#))
- Python package upgrades
 - accelerate 1.12.0 -> 1.13.0
 - bigframes 2.35.0 -> 2.38.0
 - blosc2 4.0.0 -> 4.1.2
 - diffusers 0.36.0 -> 0.37.0
 - google-adk 1.25.1 -> 1.27.1

EXP: 2

AIM: Creating Matplotlib and ScikitLearn & Data exploration & Preprocessing in Python

PROGRAM:



This screenshot shows the first cell of a Google Colab notebook. The code imports necessary libraries (pandas, matplotlib, sklearn) and creates a DataFrame with student data. The data includes student IDs, classes attended, and internal marks. The DataFrame is printed, and the features (Classes Attended) and target variable (Internal Marks) are extracted. The data is then split into training and testing sets using `train_test_split` with a test size of 0.2 and a random state of 7.

```
[ ] [ ]
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = {

    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],

    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],

    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]

}

df = pd.DataFrame(data)

print(df)

X = df[['Classes_Attended']]

y = df['Internal_Marks']

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=7
```

This screenshot shows the second cell of the Google Colab notebook. The code continues from the first cell by creating a `LinearRegression` model, fitting it to the training data, and making predictions on the test data. The Mean Squared Error (MSE) is calculated and printed. The model's coefficients and intercept are also printed. Finally, a scatter plot is created showing the relationship between 'Classes Attended' (X-axis) and 'Internal Marks' (Y-axis), with a linear regression line overlaid. The plot is titled 'Classes Attended vs Internal Marks' and is displayed.

```
[ ] [ ]
model = LinearRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

mse = mean_squared_error(y_test, predictions)

print("MSE:", mse)

print("Marks per Class:", model.coef_[0])

print("Base Marks:", model.intercept_)

plt.scatter(X, y)

plt.plot(X, model.predict(X))

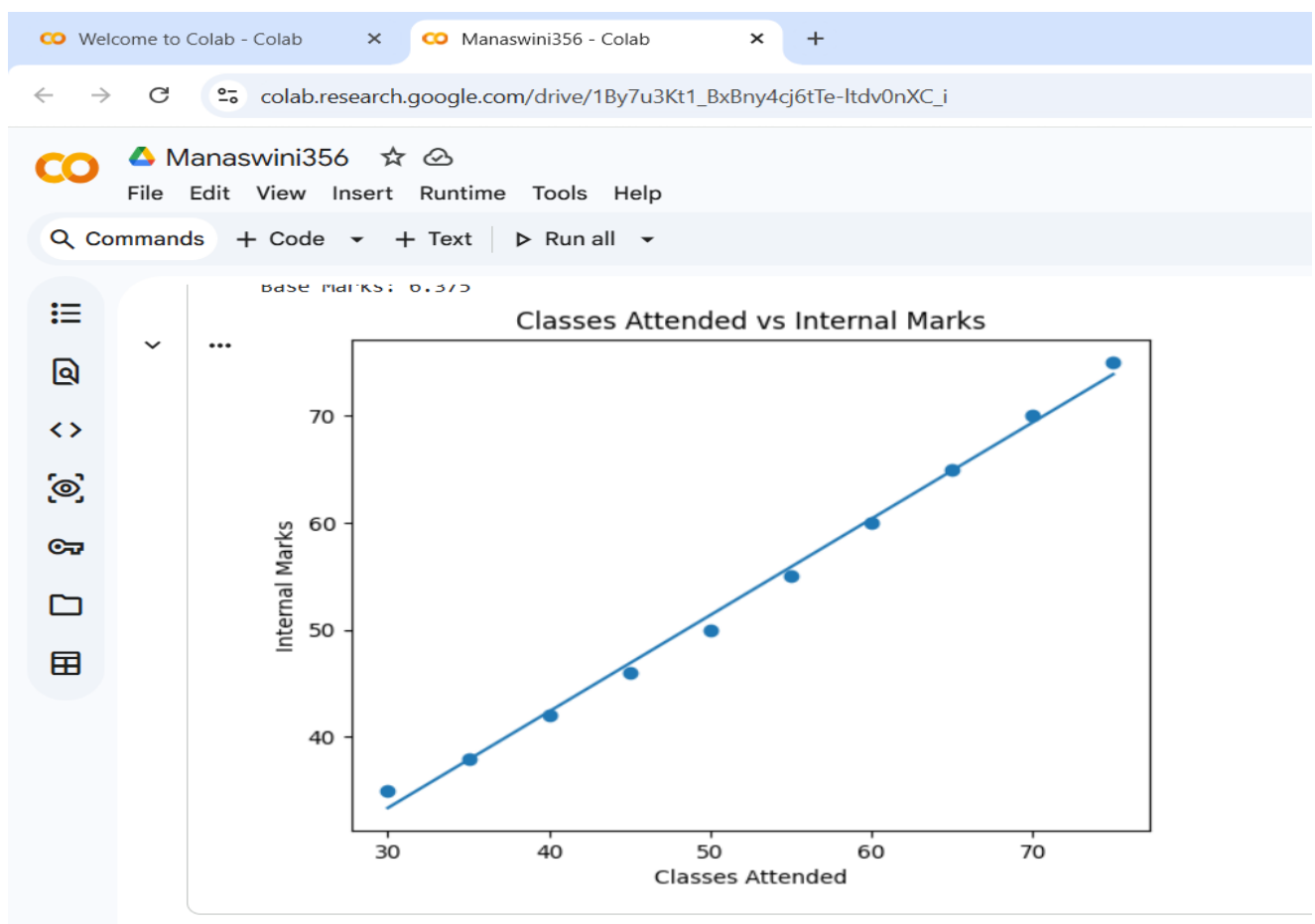
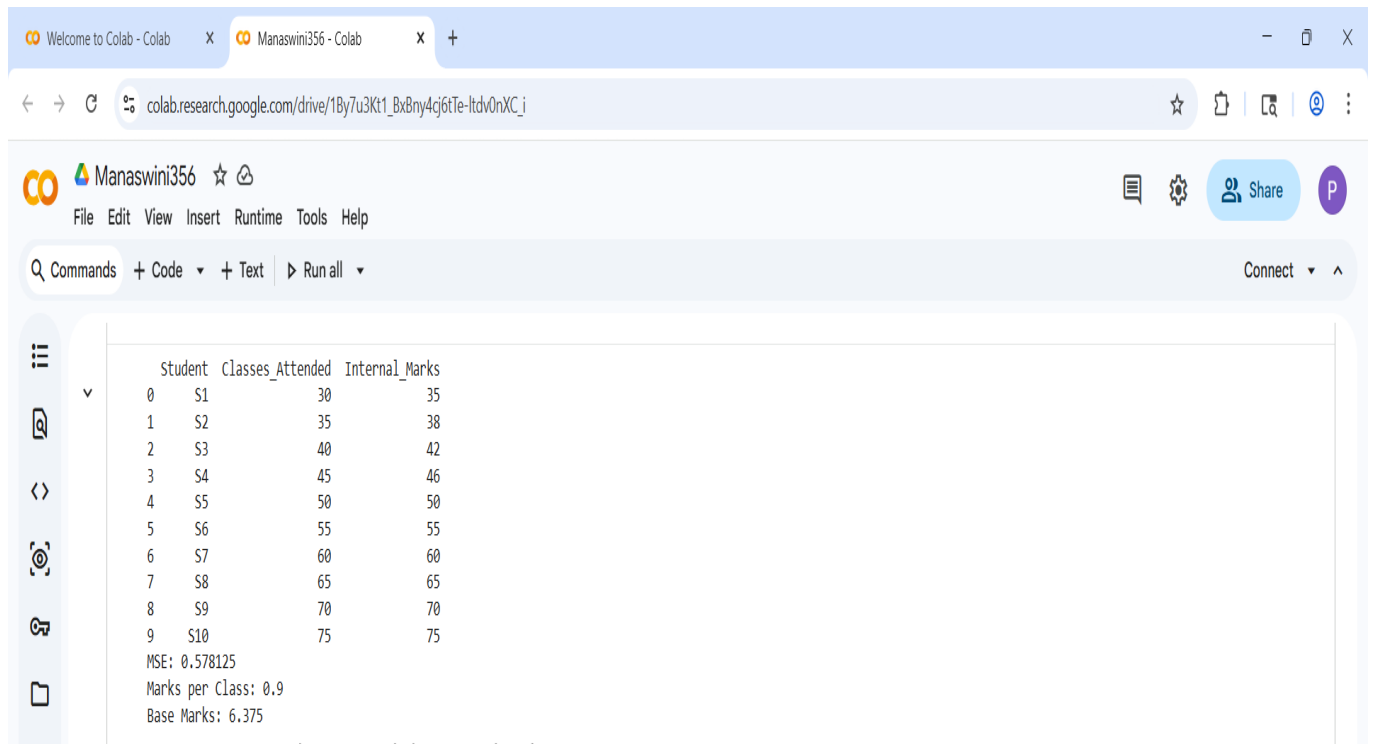
plt.xlabel("Classes Attended")

plt.ylabel("Internal Marks")

plt.title("Classes Attended vs Internal Marks")

plt.show()
```

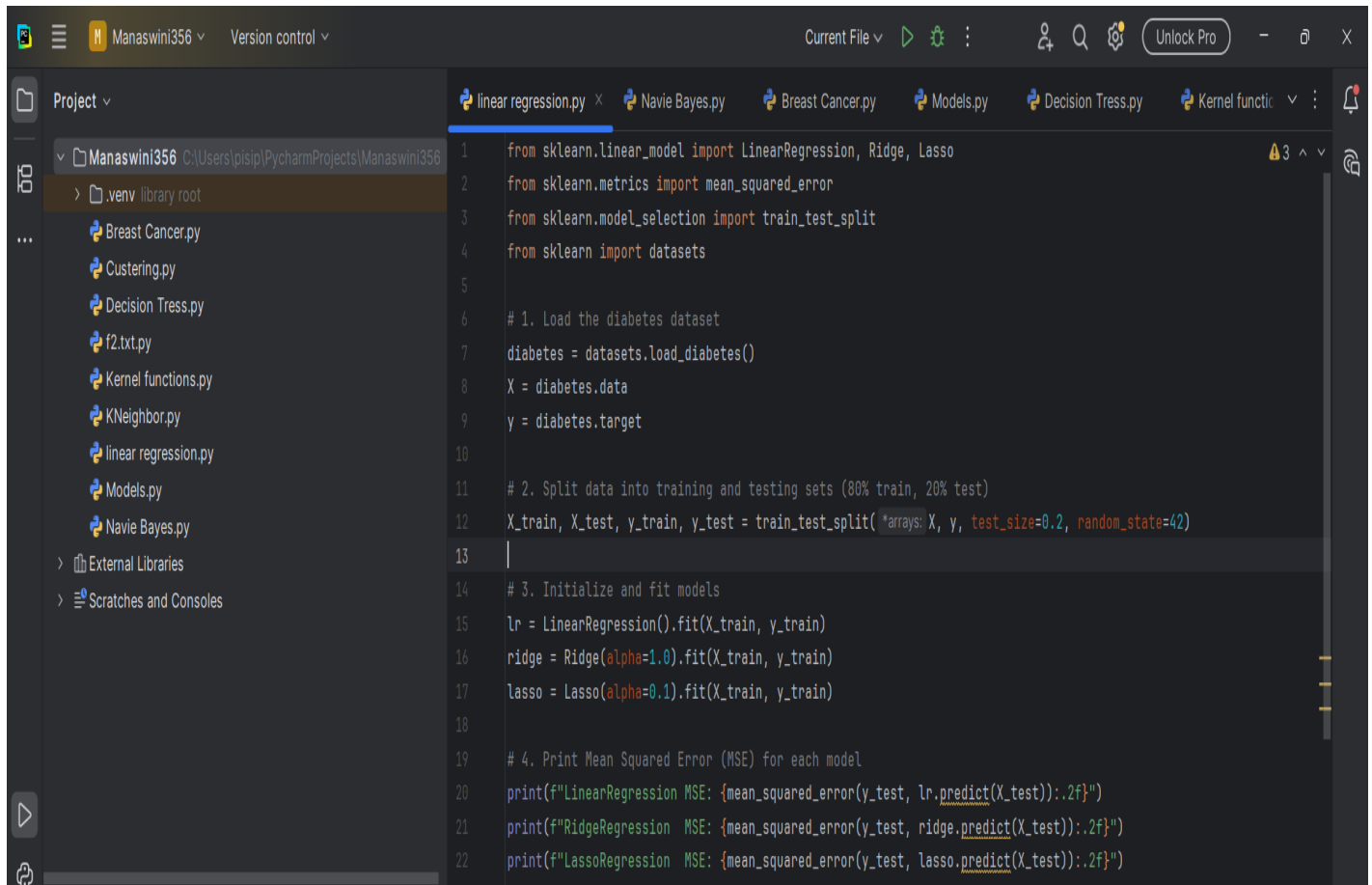
OUTPUT:



EXP: 3

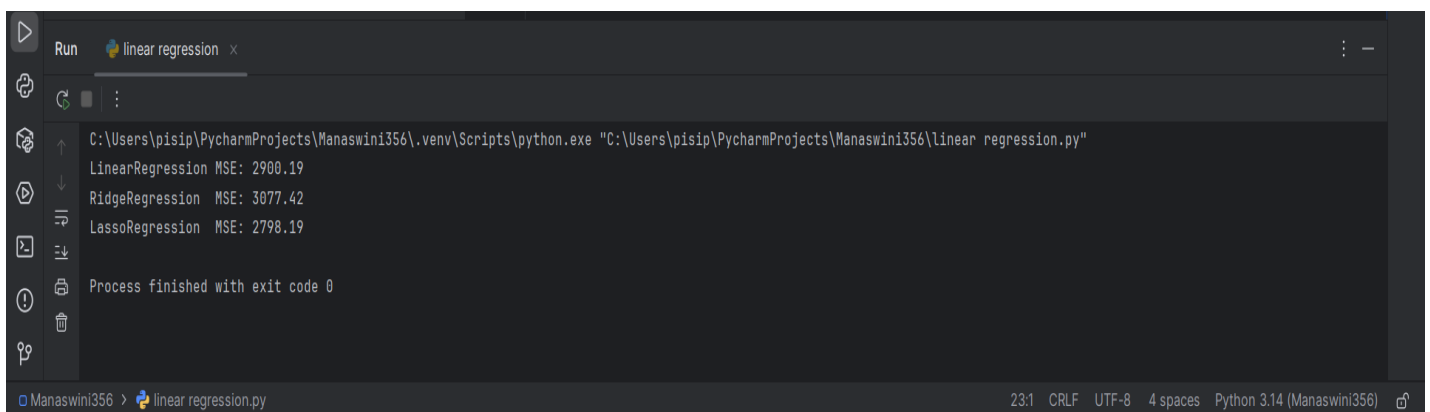
AIM: Implement Regularized Linear Regression

PROGRAM:



```
1 from sklearn.linear_model import LinearRegression, Ridge, Lasso
2 from sklearn.metrics import mean_squared_error
3 from sklearn.model_selection import train_test_split
4 from sklearn import datasets
5
6 # 1. Load the diabetes dataset
7 diabetes = datasets.load_diabetes()
8 X = diabetes.data
9 y = diabetes.target
10
11 # 2. Split data into training and testing sets (80% train, 20% test)
12 X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2, random_state=42)
13
14 # 3. Initialize and fit models
15 lr = LinearRegression().fit(X_train, y_train)
16 ridge = Ridge(alpha=1.0).fit(X_train, y_train)
17 lasso = Lasso(alpha=0.1).fit(X_train, y_train)
18
19 # 4. Print Mean Squared Error (MSE) for each model
20 print(f"LinearRegression MSE: {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
21 print(f"RidgeRegression MSE: {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
22 print(f"LassoRegression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
```

OUTPUT:



```
Run linear regression x
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisip\PycharmProjects\Manaswini356\linear_regression.py"
LinearRegression MSE: 2900.19
RidgeRegression MSE: 3077.42
LassoRegression MSE: 2798.19
Process finished with exit code 0
```

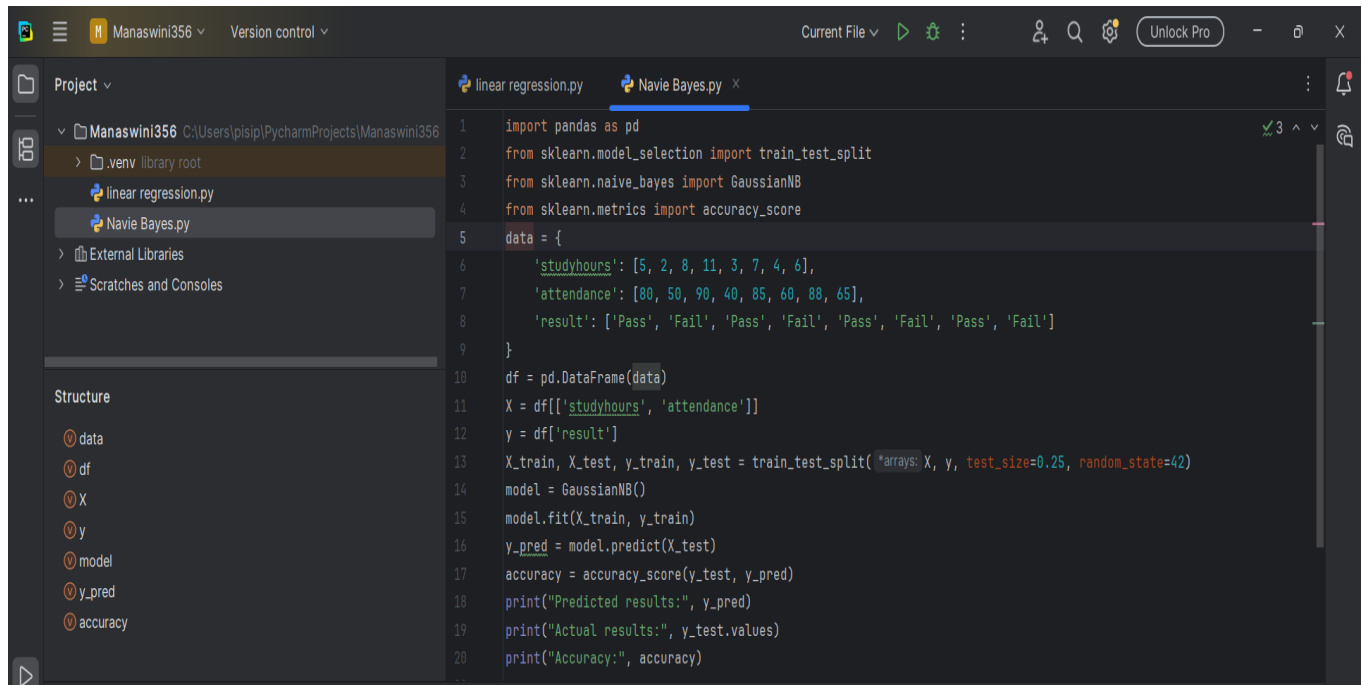
RESULT:

The model was successfully implemented and regularization helped reduce overfitting and improved prediction accuracy.

EXP: 4

AIM: Implement Navie Bayes Classifier

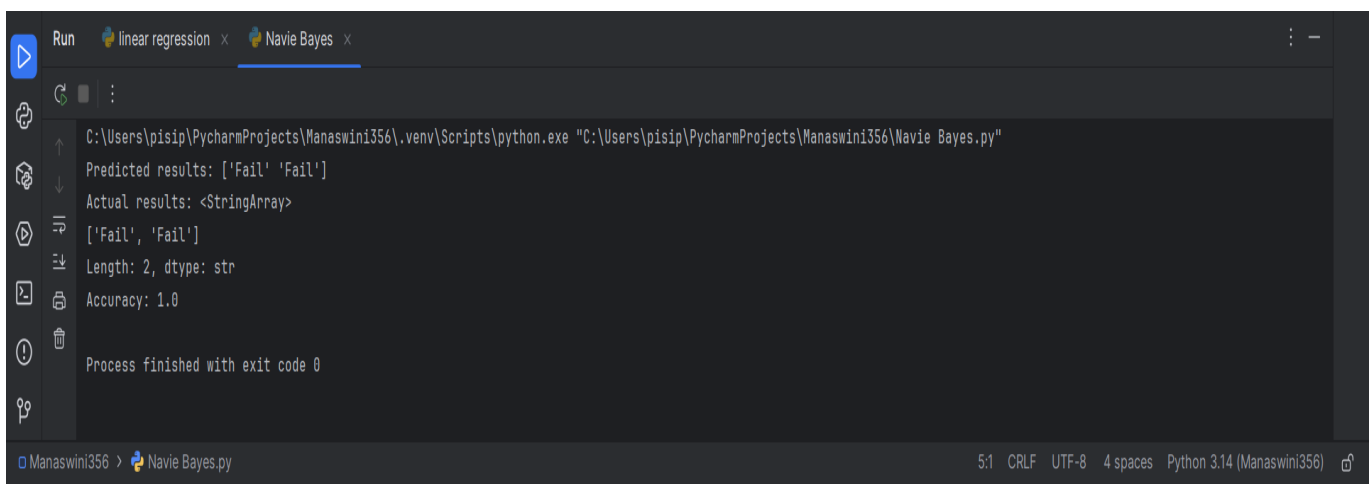
PROGRAM:



The screenshot shows the PyCharm IDE with a project named 'Manaswini356'. The file explorer on the left shows the project structure, including a '.venv' directory and two Python files: 'linear regression.py' and 'Navie Bayes.py'. The 'Navie Bayes.py' file is open in the editor, showing the following code:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.naive_bayes import GaussianNB
4 from sklearn.metrics import accuracy_score
5 data = {
6     'studyhours': [5, 2, 8, 11, 3, 7, 4, 6],
7     'attendance': [80, 50, 90, 40, 85, 60, 88, 65],
8     'result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
9 }
10 df = pd.DataFrame(data)
11 X = df[['studyhours', 'attendance']]
12 y = df['result']
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
14 model = GaussianNB()
15 model.fit(X_train, y_train)
16 y_pred = model.predict(X_test)
17 accuracy = accuracy_score(y_test, y_pred)
18 print("Predicted results:", y_pred)
19 print("Actual results:", y_test.values)
20 print("Accuracy:", accuracy)
```

OUTPUT:



The screenshot shows the PyCharm Run console with the following output:

```
Run C:\Users\pisp\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisp\PycharmProjects\Manaswini356\Navie Bayes.py"
Predicted results: ['Fail' 'Fail']
Actual results: <StringArray>
['Fail', 'Fail']
Length: 2, dtype: str
Accuracy: 1.0
Process finished with exit code 0
```

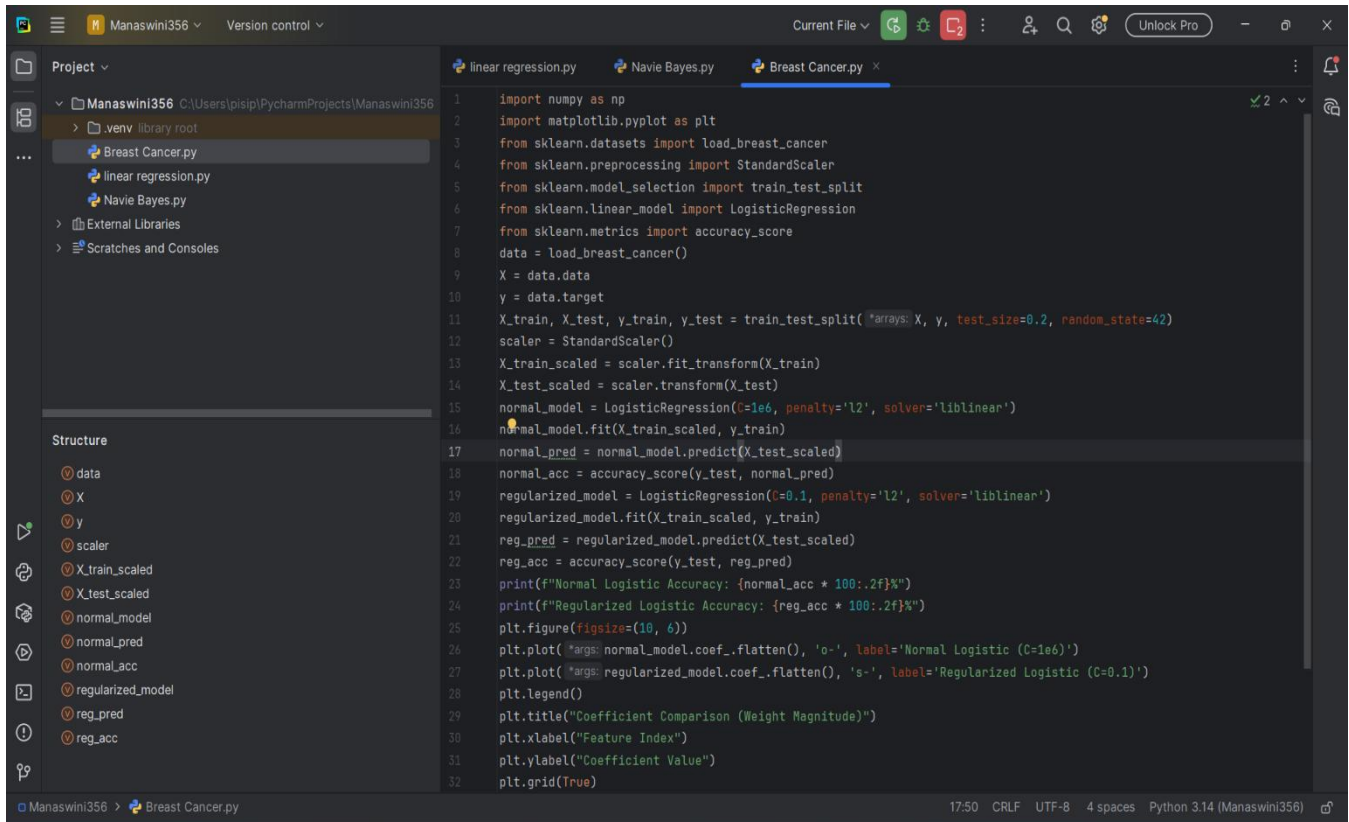
RESULT:

The Navie Bayes effectively classified the data with good accuracy.

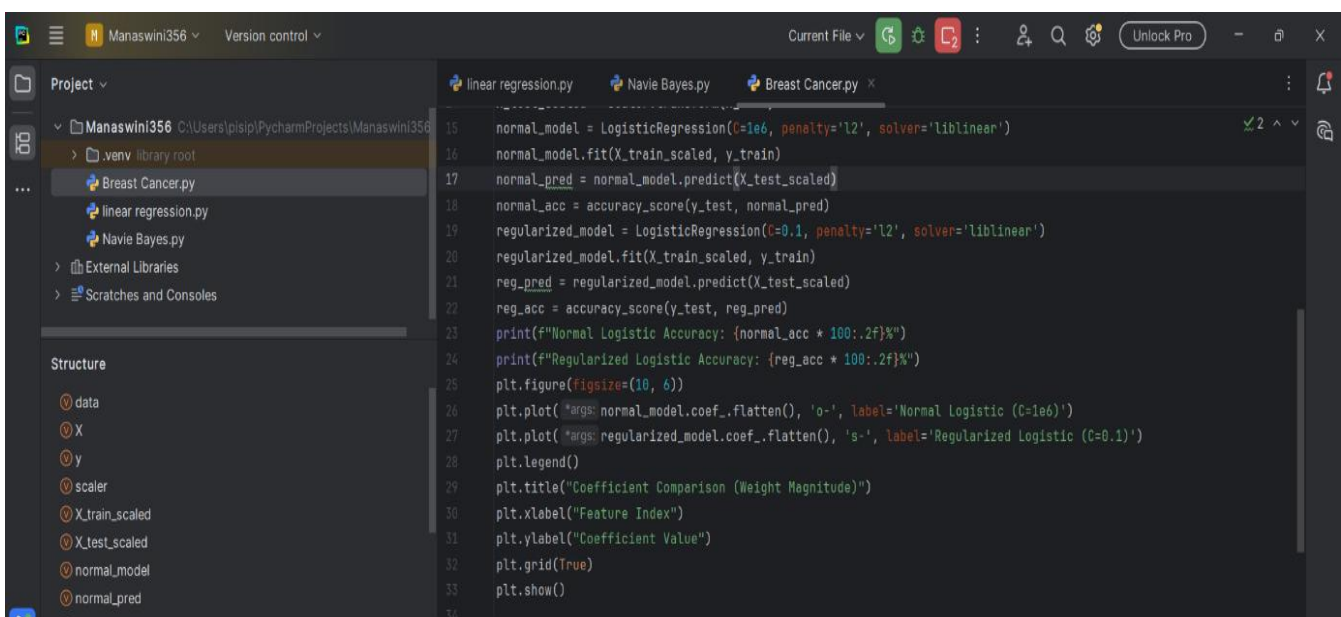
EXP: 5

AIM: Implement Regularized Logistic Regression

PROGRAM:



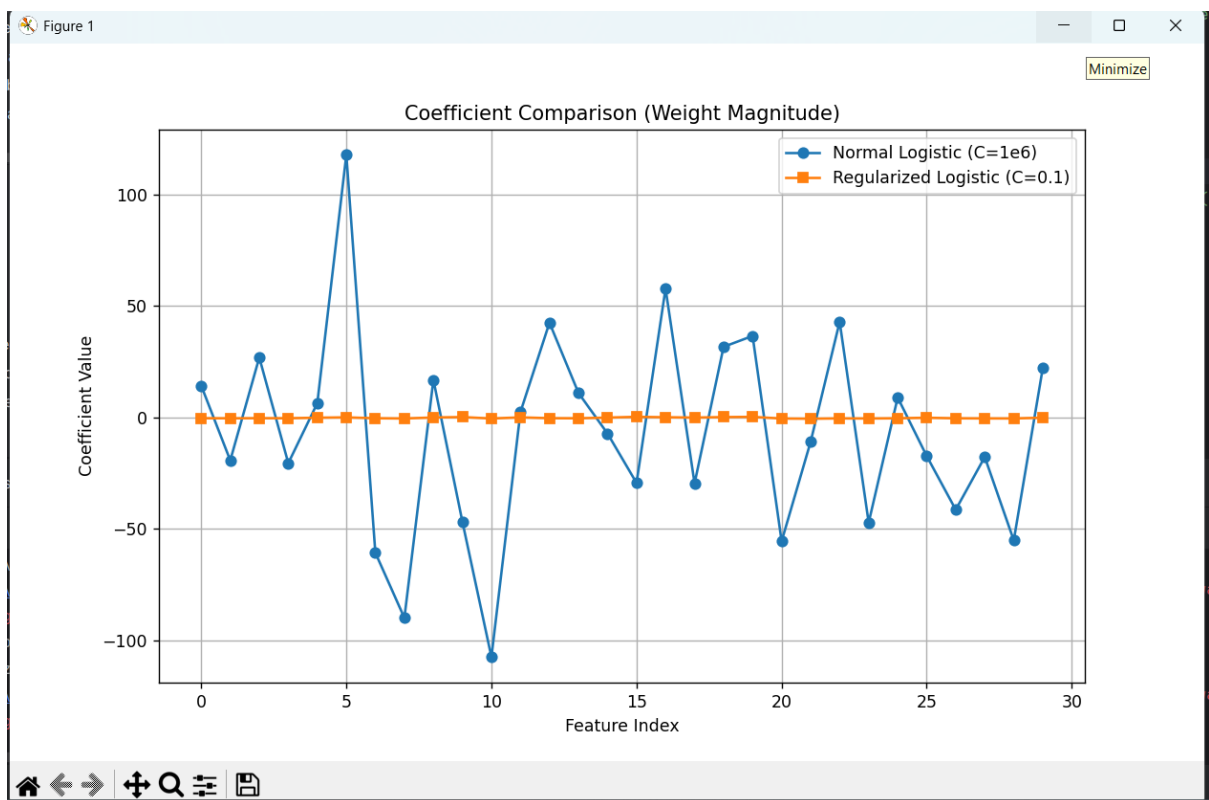
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import accuracy_score
8 data = load_breast_cancer()
9 X = data.data
10 y = data.target
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12 scaler = StandardScaler()
13 X_train_scaled = scaler.fit_transform(X_train)
14 X_test_scaled = scaler.transform(X_test)
15 normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
16 normal_model.fit(X_train_scaled, y_train)
17 normal_pred = normal_model.predict(X_test_scaled)
18 normal_acc = accuracy_score(y_test, normal_pred)
19 regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
20 regularized_model.fit(X_train_scaled, y_train)
21 reg_pred = regularized_model.predict(X_test_scaled)
22 reg_acc = accuracy_score(y_test, reg_pred)
23 print(f"Normal Logistic Accuracy: {normal_acc * 100:.2f}%")
24 print(f"Regularized Logistic Accuracy: {reg_acc * 100:.2f}%")
25 plt.figure(figsize=(10, 6))
26 plt.plot(*args: normal_model.coef_.flatten(), 'o-', label='Normal Logistic (C=1e6)')
27 plt.plot(*args: regularized_model.coef_.flatten(), 's-', label='Regularized Logistic (C=0.1)')
28 plt.legend()
29 plt.title("Coefficient Comparison (Weight Magnitude)")
30 plt.xlabel("Feature Index")
31 plt.ylabel("Coefficient Value")
32 plt.grid(True)
```



```
15 normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
16 normal_model.fit(X_train_scaled, y_train)
17 normal_pred = normal_model.predict(X_test_scaled)
18 normal_acc = accuracy_score(y_test, normal_pred)
19 regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
20 regularized_model.fit(X_train_scaled, y_train)
21 reg_pred = regularized_model.predict(X_test_scaled)
22 reg_acc = accuracy_score(y_test, reg_pred)
23 print(f"Normal Logistic Accuracy: {normal_acc * 100:.2f}%")
24 print(f"Regularized Logistic Accuracy: {reg_acc * 100:.2f}%")
25 plt.figure(figsize=(10, 6))
26 plt.plot(*args: normal_model.coef_.flatten(), 'o-', label='Normal Logistic (C=1e6)')
27 plt.plot(*args: regularized_model.coef_.flatten(), 's-', label='Regularized Logistic (C=0.1)')
28 plt.legend()
29 plt.title("Coefficient Comparison (Weight Magnitude)")
30 plt.xlabel("Feature Index")
31 plt.ylabel("Coefficient Value")
32 plt.grid(True)
33 plt.show()
34
```

OUTPUT:

```
Run Breast Cancer x Breast Cancer x
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisip\PycharmProjects\Manaswini356\Breast Cancer.py"
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\lib\site-packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in version 1.9:
  warnings.warn(
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\lib\site-packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in version 1.9:
  warnings.warn(
Normal Logistic Accuracy: 93.86%
Regularized Logistic Accuracy: 99.12%
```



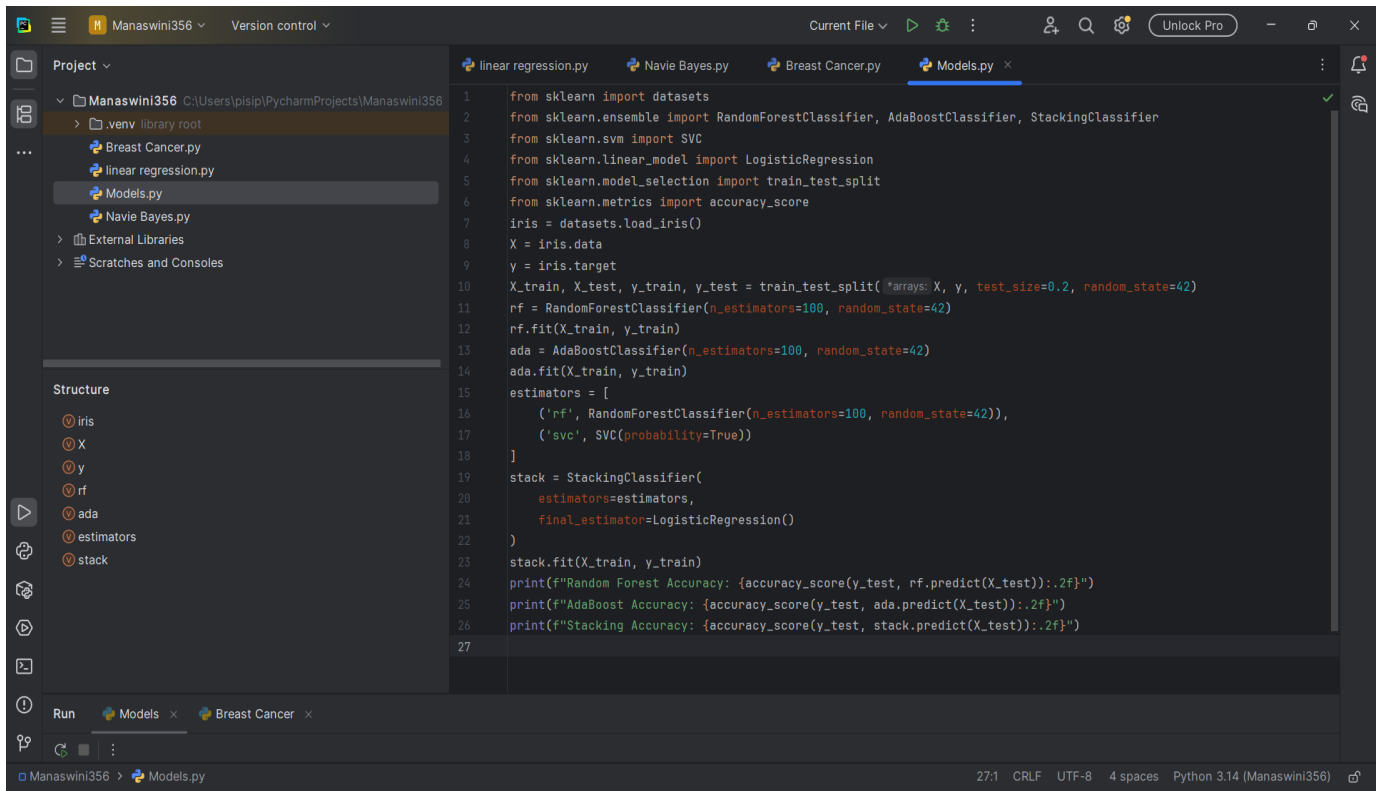
RESULT:

The model was successfully implemented and regularization improved accuracy and prevented overfitting.

EXP: 6

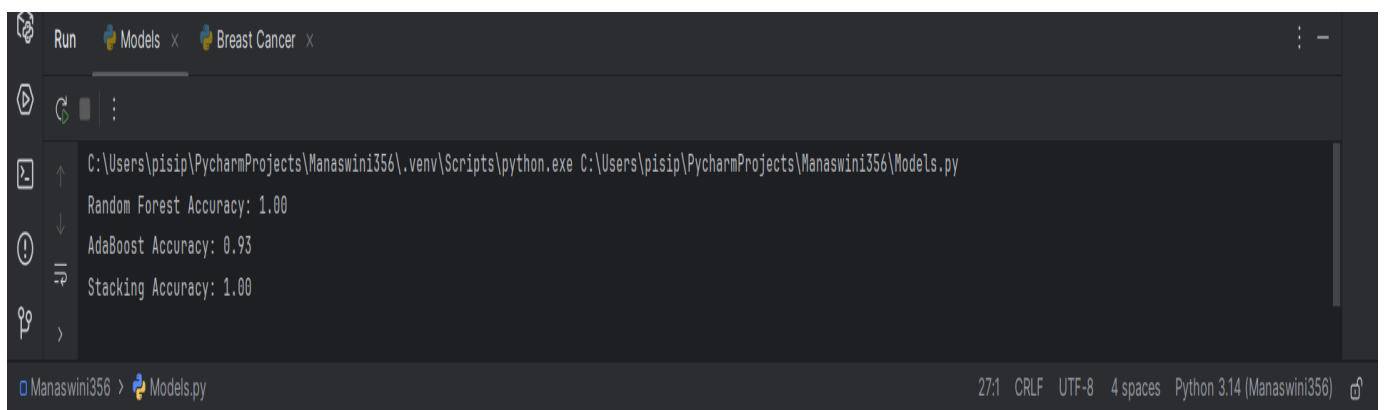
AIM: To Build Models using different Ensembling Techniques

PROGRAM:



```
1 from sklearn import datasets
2 from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
3 from sklearn.svm import SVC
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7 iris = datasets.load_iris()
8 X = iris.data
9 y = iris.target
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
11 rf = RandomForestClassifier(n_estimators=100, random_state=42)
12 rf.fit(X_train, y_train)
13 ada = AdaBoostClassifier(n_estimators=100, random_state=42)
14 ada.fit(X_train, y_train)
15 estimators = [
16     ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
17     ('svc', SVC(probability=True))
18 ]
19 stack = StackingClassifier(
20     estimators=estimators,
21     final_estimator=LogisticRegression()
22 )
23 stack.fit(X_train, y_train)
24 print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
25 print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
26 print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
27
```

OUTPUT:



```
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe C:\Users\pisip\PycharmProjects\Manaswini356\Models.py
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

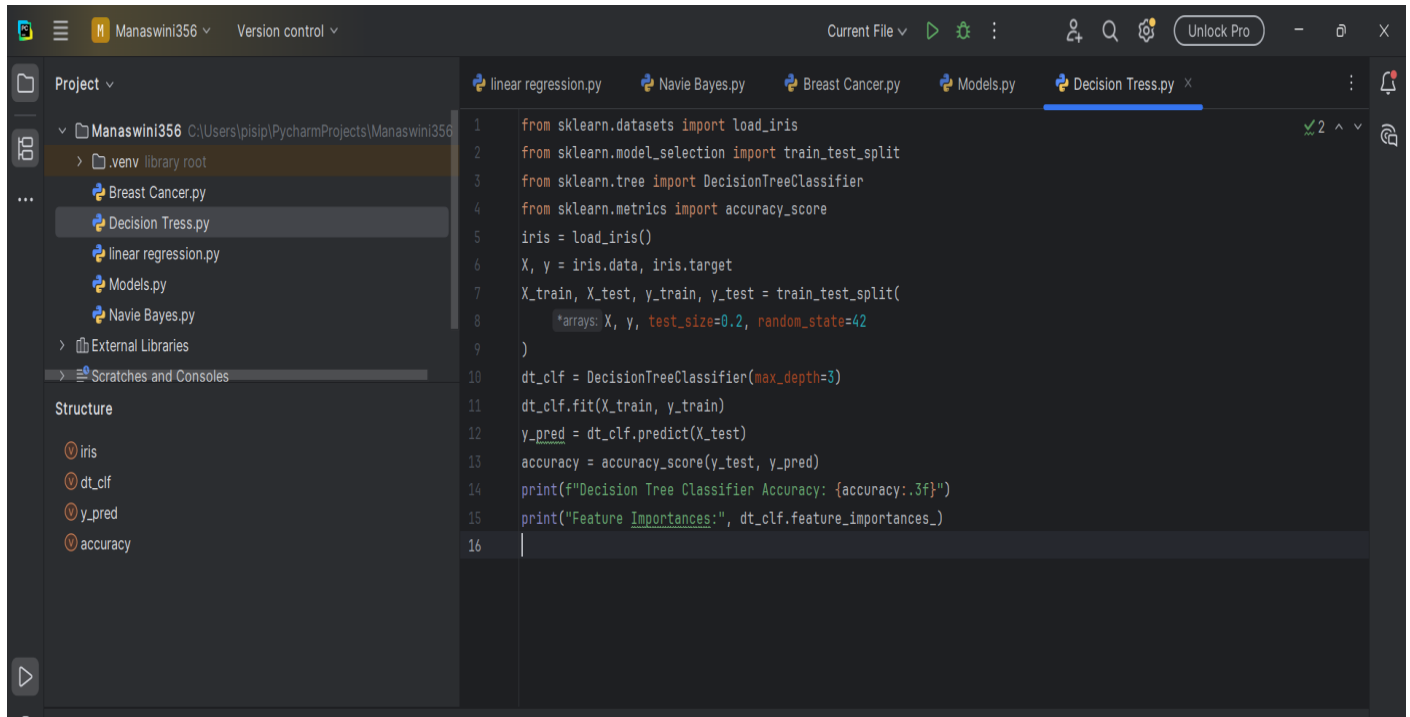
RESULT:

The ensemble models were successfully implemented and they provided better accuracy compared to indival models.

EXP: 7

AIM: To Build Models using Decision Tress

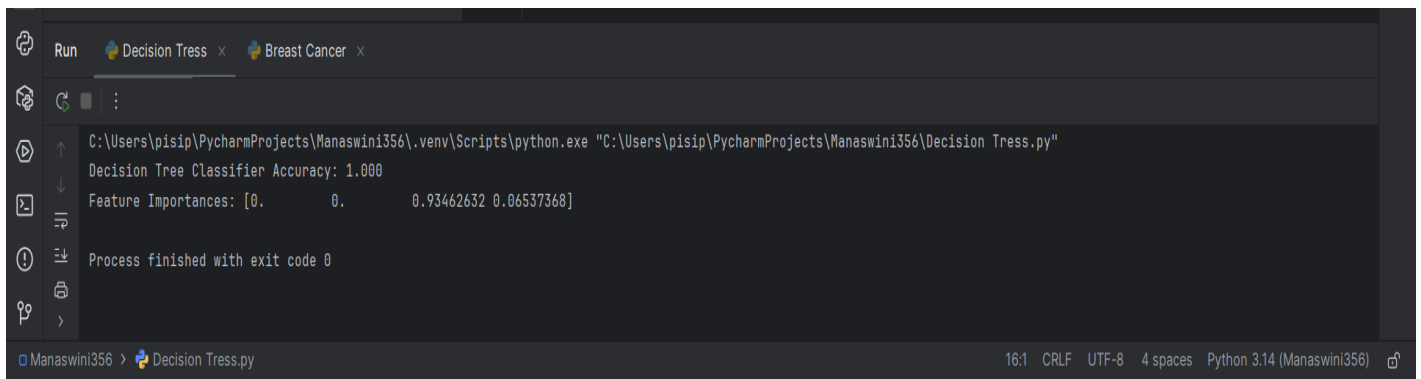
POGRAM:



The screenshot shows the PyCharm IDE interface. The left sidebar displays the project structure for 'Manaswini356', including files like 'Breast Cancer.py', 'Decision Tress.py', 'linear regression.py', 'Models.py', and 'Navie Bayes.py'. The 'Structure' panel on the left lists variables: 'iris', 'dt_clf', 'y_pred', and 'accuracy'. The main editor window shows the code for 'Decision Tress.py'.

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score
5 iris = load_iris()
6 X, y = iris.data, iris.target
7 X_train, X_test, y_train, y_test = train_test_split(
8     *arrays: X, y, test_size=0.2, random_state=42
9 )
10 dt_clf = DecisionTreeClassifier(max_depth=3)
11 dt_clf.fit(X_train, y_train)
12 y_pred = dt_clf.predict(X_test)
13 accuracy = accuracy_score(y_test, y_pred)
14 print(f"Decision Tree Classifier Accuracy: {accuracy:.3f}")
15 print("Feature Importances:", dt_clf.feature_importances_)
16
```

OUTPUT:



The screenshot shows the PyCharm Run console. The top bar indicates the running process is 'Decision Tress'. The console output shows the execution of the script, including the path to the Python interpreter and the output of the print statements.

```
Run Decision Tress x Breast Cancer x
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisip\PycharmProjects\Manaswini356\Decision Tress.py"
Decision Tree Classifier Accuracy: 1.000
Feature Importances: [0. 0. 0.93462632 0.06537368]
Process finished with exit code 0
```

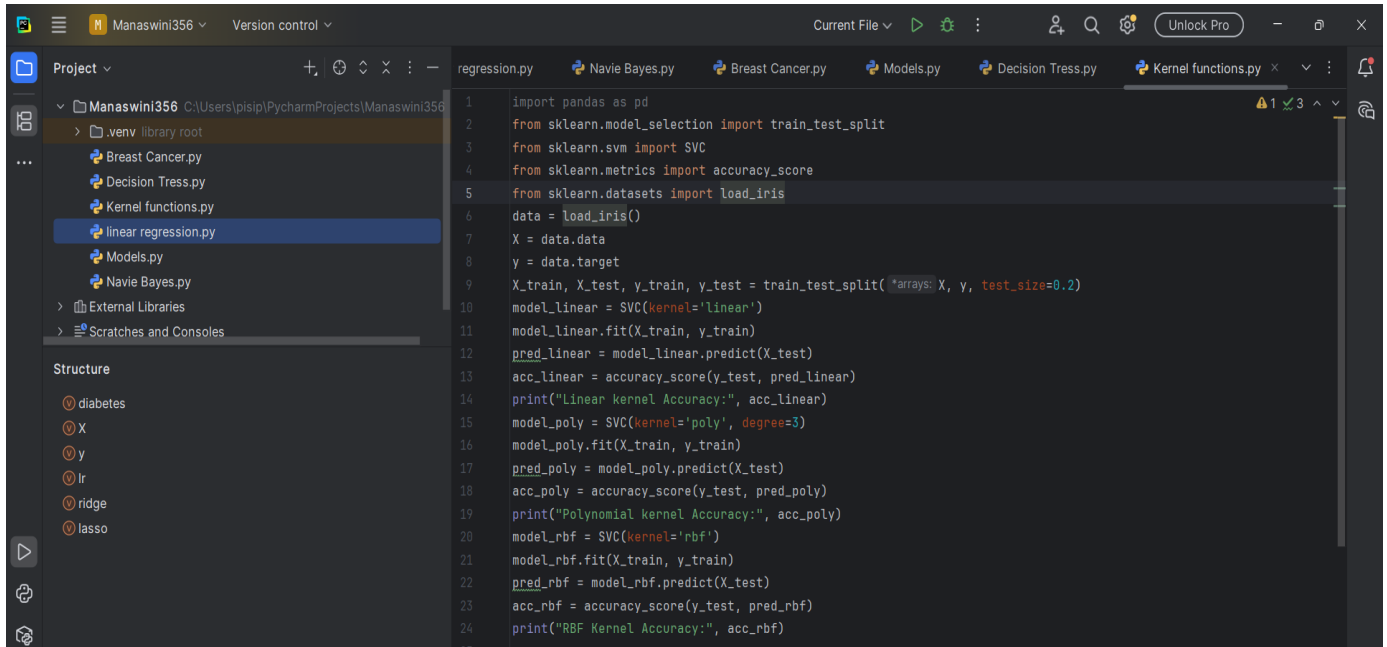
RESULT:

The Decision Tree model was successfully implemented and provided good prediction performance.

EXP: 8

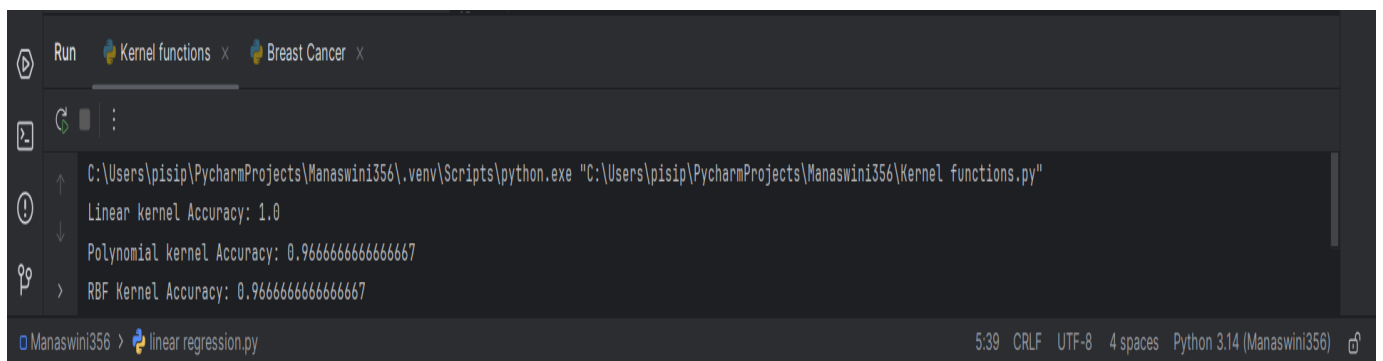
AIM: Build model using SVM with different Kernels

PROGRAM:



```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.svm import SVC
4 from sklearn.metrics import accuracy_score
5 from sklearn.datasets import load_iris
6 data = load_iris()
7 X = data.data
8 y = data.target
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
10 model_linear = SVC(kernel='linear')
11 model_linear.fit(X_train, y_train)
12 pred_linear = model_linear.predict(X_test)
13 acc_linear = accuracy_score(y_test, pred_linear)
14 print("Linear kernel Accuracy:", acc_linear)
15 model_poly = SVC(kernel='poly', degree=3)
16 model_poly.fit(X_train, y_train)
17 pred_poly = model_poly.predict(X_test)
18 acc_poly = accuracy_score(y_test, pred_poly)
19 print("Polynomial kernel Accuracy:", acc_poly)
20 model_rbf = SVC(kernel='rbf')
21 model_rbf.fit(X_train, y_train)
22 pred_rbf = model_rbf.predict(X_test)
23 acc_rbf = accuracy_score(y_test, pred_rbf)
24 print("RBF Kernel Accuracy:", acc_rbf)
```

OUTPUT:



```
Run Kernel functions x Breast Cancer x
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisip\PycharmProjects\Manaswini356\Kernel functions.py"
Linear kernel Accuracy: 1.0
Polynomial kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667
```

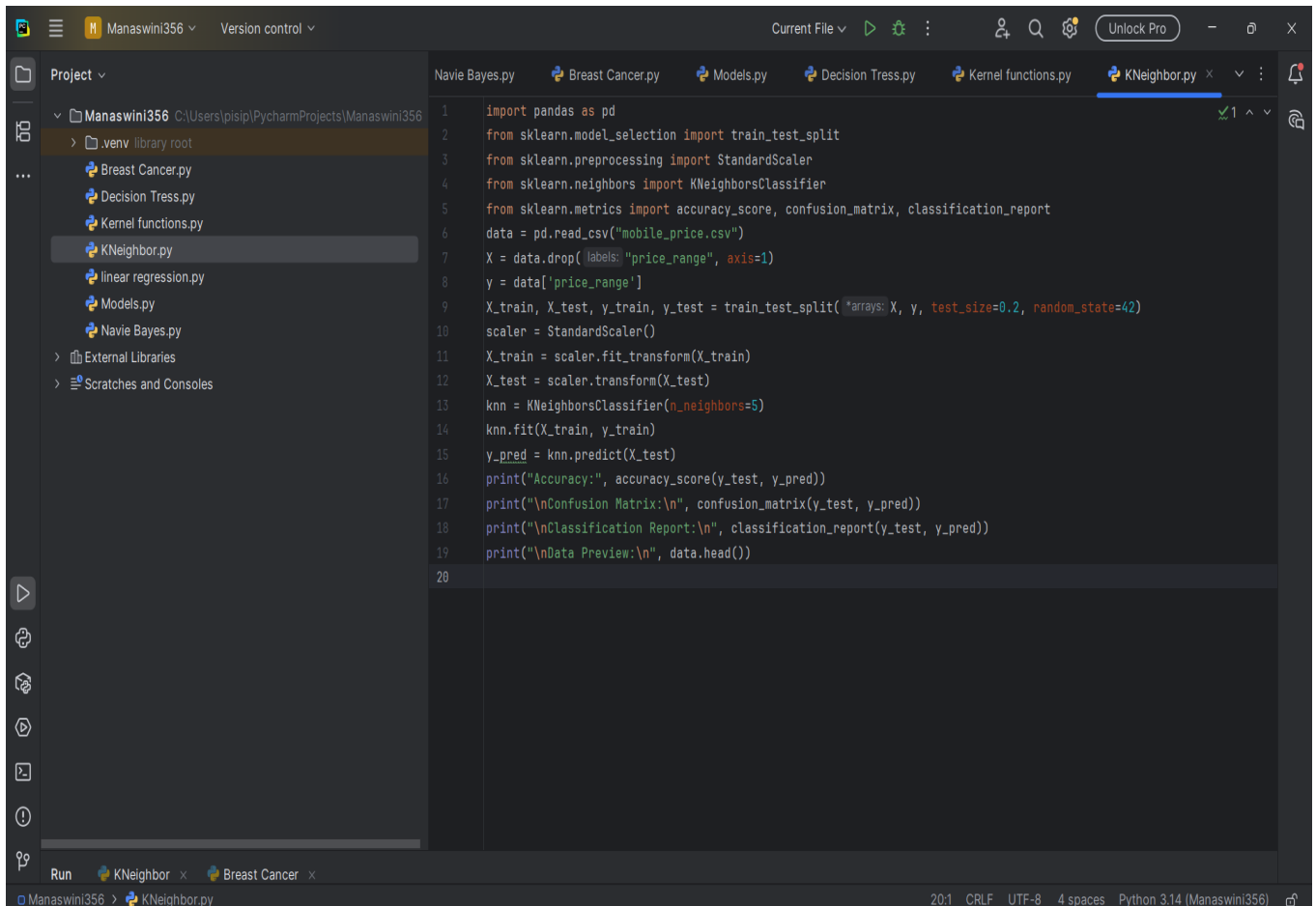
RESULT:

The SVM models were successfully implemented with various kernels and they achieved good classification performance.

EXP: 9

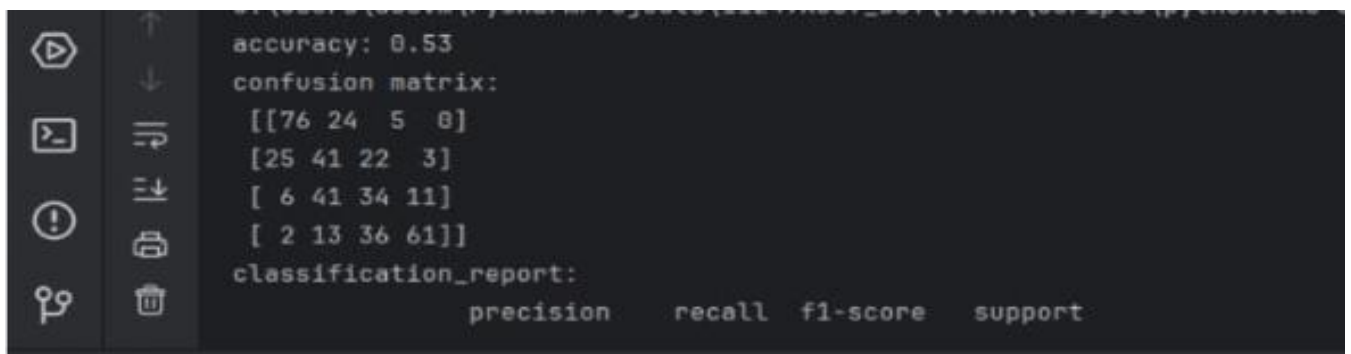
AIM: Implement K-NN algorithm to classify a dataset

PROGRAM:



```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.neighbors import KNeighborsClassifier
5 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
6 data = pd.read_csv("mobile_price.csv")
7 X = data.drop(labels="price_range", axis=1)
8 y = data['price_range']
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
10 scaler = StandardScaler()
11 X_train = scaler.fit_transform(X_train)
12 X_test = scaler.transform(X_test)
13 knn = KNeighborsClassifier(n_neighbors=5)
14 knn.fit(X_train, y_train)
15 y_pred = knn.predict(X_test)
16 print("Accuracy:", accuracy_score(y_test, y_pred))
17 print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
18 print("\nClassification Report:\n", classification_report(y_test, y_pred))
19 print("\nData Preview:\n", data.head())
20
```

OUTPUT:



```
accuracy: 0.53
confusion matrix:
[[76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]
classification_report:
               precision    recall  f1-score   support
```

RESULT:

The K-NN model was successfully implemented and it is classified.

EXP: 10

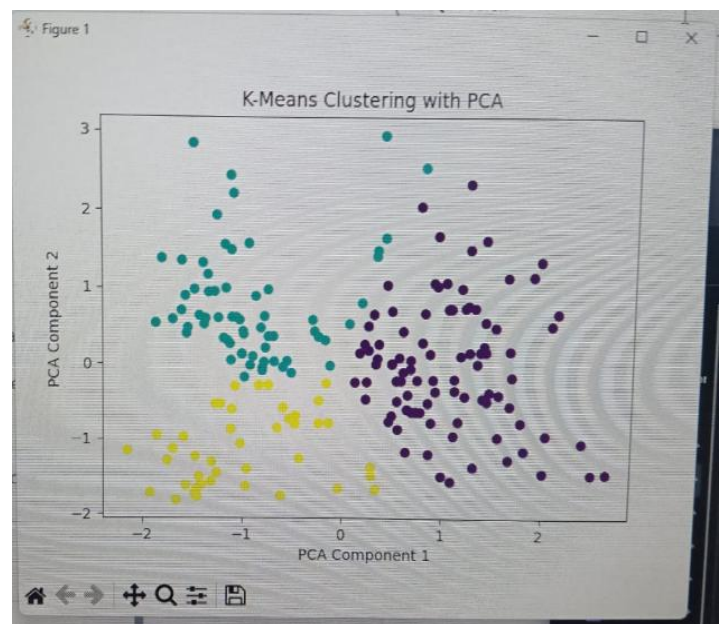
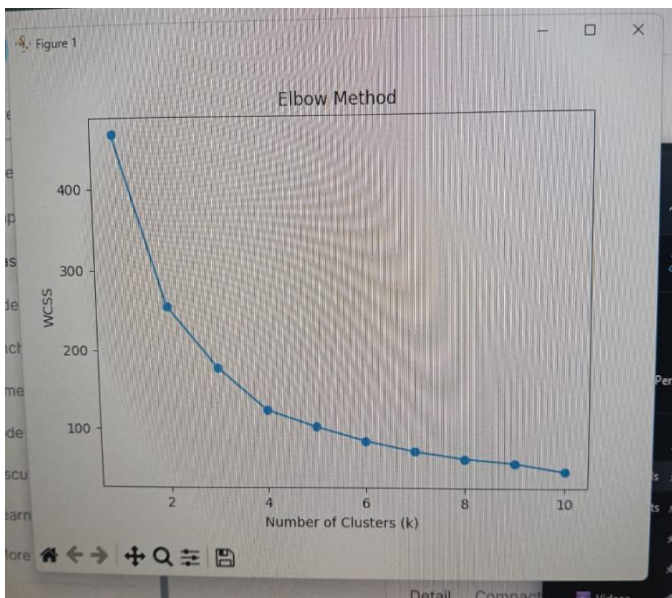
AIM: To build a K-Means Clustering with PCA and Elbow method

PROGRAM:

```
Project ▾
  ▾ Manaswini356 C:\Users\pisip\PycharmProjects\Manaswini356
    ▾ .venv library root
      Breast Cancer.py
      Clustering.py
      Decision Tress.py
      f2.txt.py
      Kernel functions.py
      KNeighbor.py
      linear regression.py
      Models.py
      Navie Bayes.py
    ▾ External Libraries
    ▾ Scratches and Consoles

dels.py Decision Tress.py Kernel functions.py KNeighbor.py Clustering.py x Mall_Customers (1).csv
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.decomposition import PCA
5 from sklearn.cluster import KMeans
6 data = pd.read_csv("Mall_Customers(1).csv")
7 X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
8 scaler = StandardScaler()
9 X_scaled = scaler.fit_transform(X)
10 pca = PCA(n_components=2)
11 X_pca = pca.fit_transform(X_scaled)
12 print("Explained variance ratio:", pca.explained_variance_ratio_)
13 wcss = []
14 for i in range(1, 11):
15     kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
16     kmeans.fit(X_pca)
17     wcss.append(kmeans.inertia_)
18 plt.figure(figsize=(8, 5))
19 plt.plot(range(1, 11), wcss, marker='o')
20 plt.title("Elbow Method")
21 plt.xlabel("Number of Clusters (K)")
22 plt.ylabel("WCSS")
23 plt.show()
24 kmeans = KMeans(n_clusters=5, random_state=0, n_init=10)
25 clusters = kmeans.fit_predict(X_pca)
26 plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
27 plt.title("K-means Clustering with PCA")
28 plt.xlabel("PCA Component 1")
29 plt.ylabel("PCA Component 2")
30 plt.show()
```

OUTPUT:



RESULT:

The K-Means Clustering was successfully implemented after PCA and Elbow method and the data was grouped effectively into clusters.